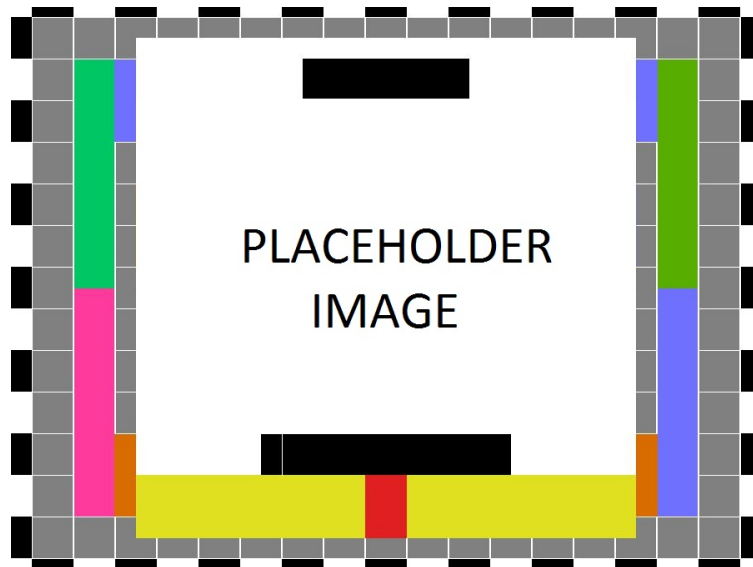


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SPRING 2027**



**TEAM TRAILBLAZER
TERRASCOUT AUTONOMOUS GROUND ROVER**

ALAN TURING
GRACE HOPPER
JOHN VON NEUMANN
ADA LOVELACE
CHARLES BABBAGE

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	01.20.2027	CB	document creation
1.0	02.05.2027	AT, GH, CB	official release

CONTENTS

1	Introduction	4
2	Design Stakeholders & Concerns	4
3	Design Viewpoints	4
4	System Overview	4
5	X Layer Subsystems — Perception & Sensing	4
5.1	Layer Resources	4
5.2	X.Imaging	4
6	Y Layer Subsystems — Autonomy & Compute	4
6.1	Layer Resources	4
6.2	Y.Navigation	4
6.3	Y.FaultDetection	4
6.4	Y.Mission & Y.GroundStationServer	5
7	Z Layer Subsystems — Mobility & Power	6
7.1	Layer Resources	6
7.2	Z.Drive	6
7.3	Z.Power	6
8	Requirements Traceability	6
9	Appendix A	6

1 INTRODUCTION

This DDS gives the implementation-level design of TerraScout Autonomous Ground Rover as a Software Design Description per IEEE Std 1016-2009 [1]. It builds on the SRS (requirements) and ADS (architecture, three layers X/Y/Z), adding hardware, software, data, and algorithm detail. Representative subsystems are detailed below.

2 DESIGN STAKEHOLDERS & CONCERNS

Implementers, integrators, testers, and maintainers. Concerns: composition, interfaces, data structures, timing/state behavior, algorithms, and the hardware/OS resources each subsystem requires.

3 DESIGN VIEWPOINTS

Each subsystem is described through the relevant IEEE 1016 viewpoints: Context, Composition, Information (data structures), Interface, Interaction & State, Algorithm, and Resource (hardware/OS/dependencies/language). Viewpoints that do not apply are omitted.

4 SYSTEM OVERVIEW

Compute runs on an NVIDIA Jetson-class module (Ubuntu, ROS 2); platform subsystems are STM32 microcontrollers on a CAN bus. The ground-station app runs on the operator laptop.

5 X LAYER SUBSYSTEMS — PERCEPTION & SENSING

5.1 LAYER RESOURCES

Hardware: RGB CSI camera, USB thermal camera, RTK-GNSS receiver, IMU. **OS:** Ubuntu 22.04 on the compute module. **Dependencies:** ROS 2, OpenCV, GNSS driver.

5.2 X.IMAGING

Context/Composition: a ROS 2 node wrapping both cameras and a hardware trigger. **Information:** `Frame{stamp, pose, rgb[H][W][3], thermal[h][w]}`. **Interface (#01):** subscribes `/trigger`, `/pose`; publishes `/frames`. **Algorithm/State:** on trigger, grab both cameras, attach nearest-in-time pose, publish. **Resource:** C++ for the driver; ~8 W. **Rationale:** hardware trigger removes RGB/thermal skew (AD-02); C++ meets frame-rate.

6 Y LAYER SUBSYSTEMS — AUTONOMY & COMPUTE

6.1 LAYER RESOURCES

OS/Deps: Ubuntu, ROS 2, Nav2 stack, PyTorch (inference), FastAPI (ground station). **Languages:** C++ (navigation), Python (detection, server).

6.2 Y.NAVIGATION

Composition: Nav2-based planner + controller nodes. **Interface (#03):** in `/pose`, `/waypoints`, `/proximity`; out `/cmd_vel`. **Algorithm:** pure-pursuit path following with a costmap for obstacle avoidance; cross-track error held ≤ 0.5 m (FR-01). **State:** PLAN→FOLLOW→AVOID→ARRIVED. **Ratio-nale:** Nav2 reuse over a hand-written planner (AD-01).

6.3 Y.FAULTDETECTION

Composition: a Python node loading a replaceable model artifact (QA-01). **Information:** input `Frame`; output `Detection{class, confidence, pose, image_ref}`. **Interface:** subscribes `/frames`; publishes `/detections`. **Algorithm:** a lightweight CNN (MobileNet-class backbone) fine-tuned on the

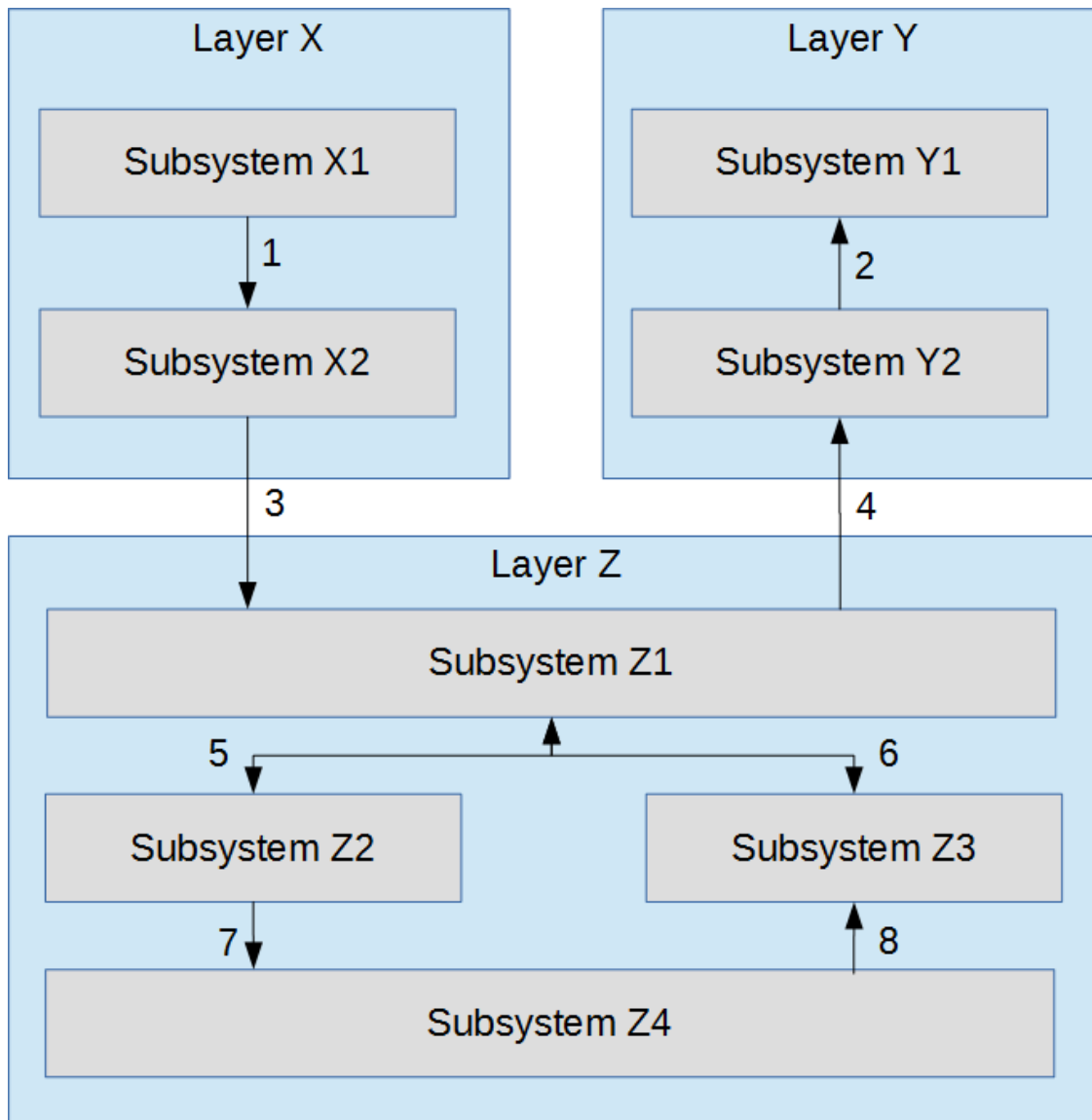


Figure 1: System architecture carried forward from the ADS

sponsor set; per-region softmax over {nominal, hotspot, soiling, crack}; thresholded to meet PE-03.
Resource: GPU inference on the Jetson. **Rationale:** edge inference per AD-02; model file swap keeps navigation untouched.

6.4 Y.MISSION & Y.GROUNDSTATIONSERVER

Y.Mission runs the state machine (idle/patrol/capture/return/dock), enforces FR-05 triggers (obstacle, SoC $\leq$ 20%, link-loss $>$ 10 s), and builds the CSV+image report (FR-04). **Y.GroundStationServer** is a FastAPI service terminating the TLS 1.3 WebSocket (SE-01, EI-01), streaming telemetry at 2 Hz and serving reports.

7 Z LAYER SUBSYSTEMS — MOBILITY & POWER

7.1 LAYER RESOURCES

Hardware: 4×brushless motors + CAN motor controllers, 24 V LiFePO4 battery + BMS, STM32 micro-controllers. **Languages:** C (firmware).

7.2 Z.DRIVE

Interface (#04): consumes /cmd_vel bridged to CAN torque commands; returns wheel odometry. **State:** a hardware E-stop line de-energizes motor power independent of firmware (SA-01). **Algorithm:** per-wheel PID velocity control. **Rationale:** hardware E-stop path is not software-defeatable.

7.3 Z.POWER

Reports state-of-charge from the BMS over CAN (drives FR-05); controls dock charging contactor (EI-02).

8 REQUIREMENTS TRACEABILITY

Req	Requirement (abbrev.)	Component	Iface
FR-01	Waypoint patrol	Y.Navigation	#03
FR-02	Imagery capture	X.Imaging	#01
FR-03	Fault detection	Y.FaultDetection	—
FR-05	Safe-stop/return	Y.Mission, Z.Power	#04
SA-01	Emergency stop	Z.Drive (HW E-stop)	#04
SE-01	Encrypted link	Y.GroundStationServer	#01(GS)

9 APPENDIX A

Wiring schematic, CAN message catalog, and chassis CAD (STEP) are attached as separate files in the closeout package.

REFERENCES

- [1] IEEE, “IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions,” Institute of Electrical and Electronics Engineers, New York, NY, USA, Tech. Rep., 2009.